

L04 REFLECTIVE JOURNAL

L04_AlexanderDebusk_ITAI1371.pdf

ITAI 1371 — Module 04: Exploratory Data Analysis

Module 04 Lab - Exploratory Data Analysis **Objective:** To learn how to explore a dataset to find patterns, anomalies, and insights before modeling. EDA is like being a detective; you are looking for clues in the data that will help you build a better model. **This lab is fully coded.** Your task is to run each cell, read the detailed explanations, understand the purpose of each visualization, and then complete the experimentation section.

Part 1: Setup and Data Loading ****What is Exploratory Data Analysis (EDA)?**** EDA is the process of using summary statistics and visualizations to understand a dataset's main characteristics. Before you can build a model, you need to understand your data. What stories does it tell? Are there errors or missing values? Are there strong relationships between variables? EDA helps answer these questions. We will use the famous Titanic dataset for this lab. It contains information about passengers and, crucially, whether they survived the disaster.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
# Seaborn is a library built on top of Matplotlib that makes creating beautiful plots easier.
# Load the dataset directly from a URL
df = pd.read_csv('https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv')
print("--- First 5 Rows ---")
print(df.head())
print("--- Basic Info ---")
# .info() is a great first command. It tells us the column names, how many non-null values are in each column, and their data type.
# Notice that 'Age' and 'Cabin' have missing values!
df.info()
```

```
--- First 5 Rows ---
   PassengerId  Survived  Pclass \
0             1         0       3
1             2         1       1
2             3         1       3
3             4         1       1
4             5         0       3

   Name                               Sex  Age  SibSp  \
0  Braund, Mr. Owen Harris             male  22.0     1
1  Cumings, Mrs. John Bradley (Florence Briggs Th...  female  38.0     1
2    Heikkinen, Miss. Laina             female  26.0     0
3  Futrelle, Mrs. Jacques Heath (Lily May Peel)    female  35.0     1
4    Allen, Mr. William Henry             male  35.0     0

   Parch  Ticket   Fare Cabin Embarked
0      0   A/5 21171   7.2500  NaN      S
1      0    PC 17599  71.2833   C85      C
2      0  STON/O2. 3101282   7.9250  NaN      S
3      0   113803   53.1000  C123      S
4      0   373450   8.0500  NaN      S

--- Basic Info ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
#   Column      Non-Null Count  Dtype
---  -
0   PassengerId  891 non-null    int64
1   Survived     891 non-null    int64
2   Pclass       891 non-null    int64
3   Name         891 non-null    object
4   Sex          891 non-null    object
5   Age          714 non-null    float64
6   SibSp        891 non-null    int64
7   Parch        891 non-null    int64
8   Ticket       891 non-null    object
```

```

9   Fare      891 non-null   float64
10  Cabin     204 non-null   object
11  Embarked  889 non-null   object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB

```

Part 2: Descriptive Statistics Let's start by getting a high-level numerical summary of the data.

- ✦ The `.describe()` method is perfect for this. It calculates statistics like mean, standard deviation, min, and max for the numerical columns.

```

# Get summary statistics for numerical columns
print("--- Descriptive Statistics ---")
print(df.describe())
print("--- Key Insights from Statistics ---")
print(f"The average age of a passenger was {df['Age'].mean():.1f} years.")
print(f"The overall survival rate was {df['Survived'].mean():.1%}.")
print(f"Fares ranged from ${df['Fare'].min()} to a whopping ${df['Fare'].max()}.")

```

```

--- Descriptive Statistics ---

```

	PassengerId	Survived	Pclass	Age	SibSp \
count	891.000000	891.000000	891.000000	714.000000	891.000000
mean	446.000000	0.383838	2.308642	29.699118	0.523008
std	257.353842	0.486592	0.836071	14.526497	1.102743
min	1.000000	0.000000	1.000000	0.420000	0.000000
25%	223.500000	0.000000	2.000000	20.125000	0.000000
50%	446.000000	0.000000	3.000000	28.000000	0.000000
75%	668.500000	1.000000	3.000000	38.000000	1.000000
max	891.000000	1.000000	3.000000	80.000000	8.000000

```


```

	Parch	Fare
count	891.000000	891.000000
mean	0.381594	32.204208
std	0.806057	49.693429
min	0.000000	0.000000
25%	0.000000	7.910400
50%	0.000000	14.454200
75%	0.000000	31.000000
max	6.000000	512.329200

```

--- Key Insights from Statistics ---
The average age of a passenger was 29.7 years.
The overall survival rate was 38.4%.
Fares ranged from $0.0 to a whopping $512.3292.

```

Part 3: Visual EDA - Telling Stories with Plots Numbers are great, but plots make patterns and relationships immediately obvious. The goal of visual EDA is to turn data into insights. **A Note on**

- ✦ **Plotting Libraries:** * **Matplotlib:** The foundational library, gives you full control over everything.* **Seaborn:** Built on Matplotlib, it provides a simpler, high-level interface for creating common statistical plots. We will use Seaborn for its ease of use and attractive defaults.

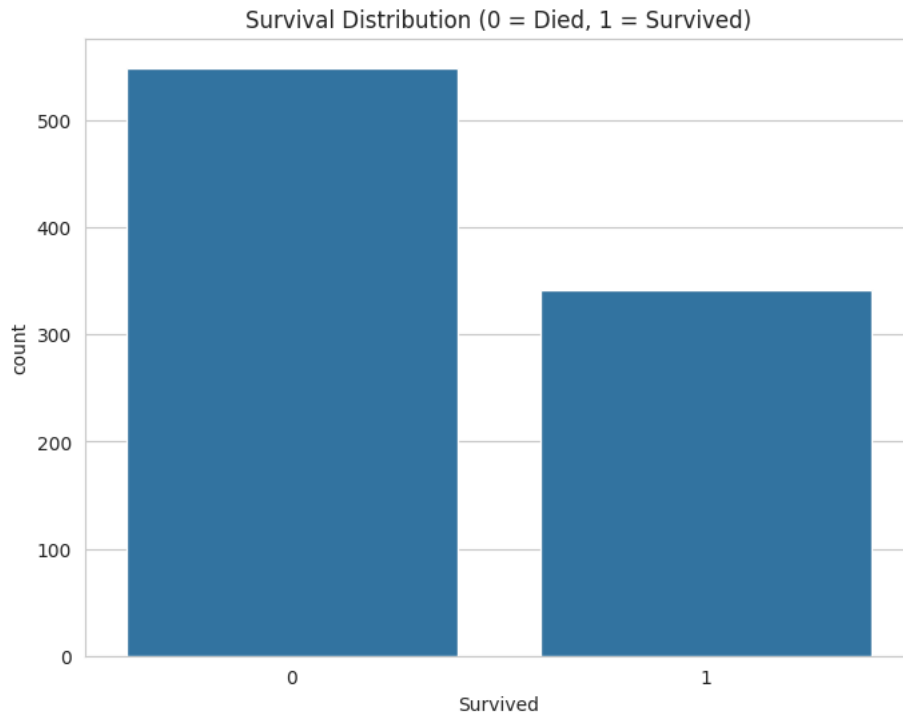
- ✦ Visualization 1: How many survived? A simple `countplot` is the best way to see the distribution of a categorical variable, like our target `Survived`.

```

sns.set_style('whitegrid') # Sets a nice visual style for our plots
plt.figure(figsize=(8, 6))
sns.countplot(x='Survived', data=df)
plt.title('Survival Distribution (0 = Died, 1 = Survived)')
plt.show()

print("Insight: Far more people died than survived. This is an example of an imbalanced dataset, which can sometimes be a challenge")

```

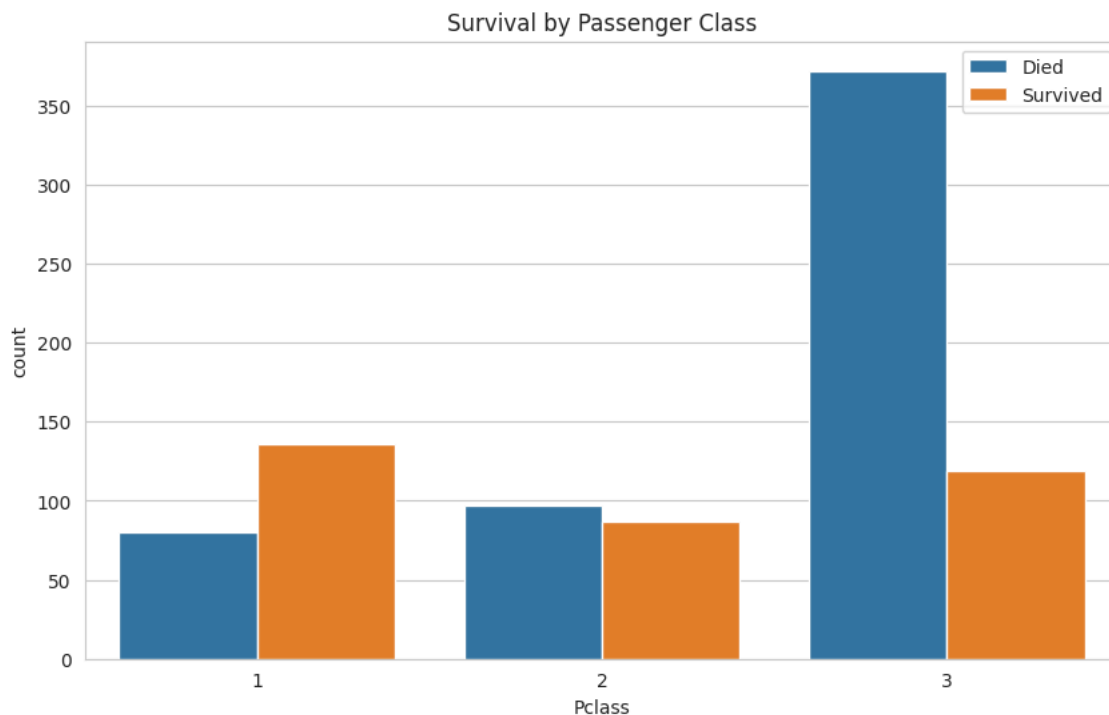


Insight: Far more people died than survived. This is an example of an imbalanced dataset, which can sometimes be a challenge for

Visualization 2: Does passenger class matter for survival? Now we want to see if there's a relationship between two variables. We can use the `hue` parameter in `countplot` to split the bars by another category.

```
plt.figure(figsize=(10, 6))
sns.countplot(x='Pclass', hue='Survived', data=df)
plt.title('Survival by Passenger Class')
plt.legend(['Died', 'Survived'])
plt.show()
```

print("Insight: This is a very strong pattern. 1st class passengers had a much higher chance of survival compared to 3rd class passengers")

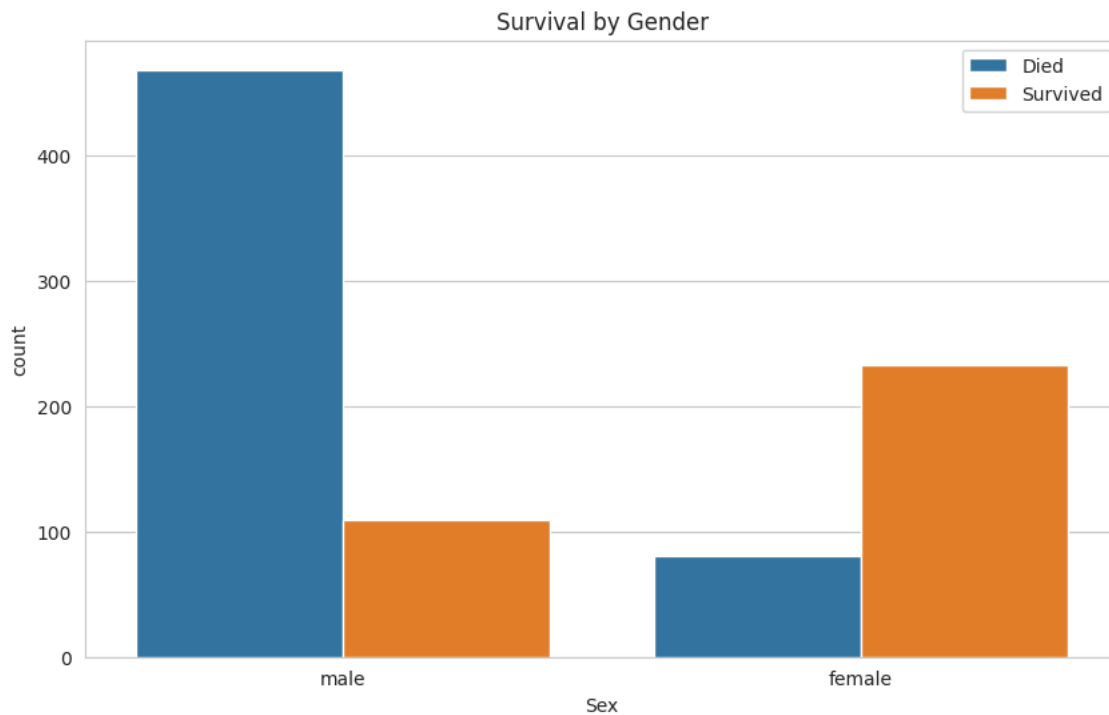


Insight: This is a very strong pattern. 1st class passengers had a much higher chance of survival compared to 3rd class passengers

- Visualization 3: What about gender? The 'women and children first' mantra is famous. Let's see if the data supports it.

```
plt.figure(figsize=(10, 6))
sns.countplot(x='Sex', hue='Survived', data=df)
plt.title('Survival by Gender')
plt.legend(['Died', 'Survived'])
plt.show()
```

print("Insight: The pattern is undeniable. A much higher proportion of females survived compared to males. This is another very strong

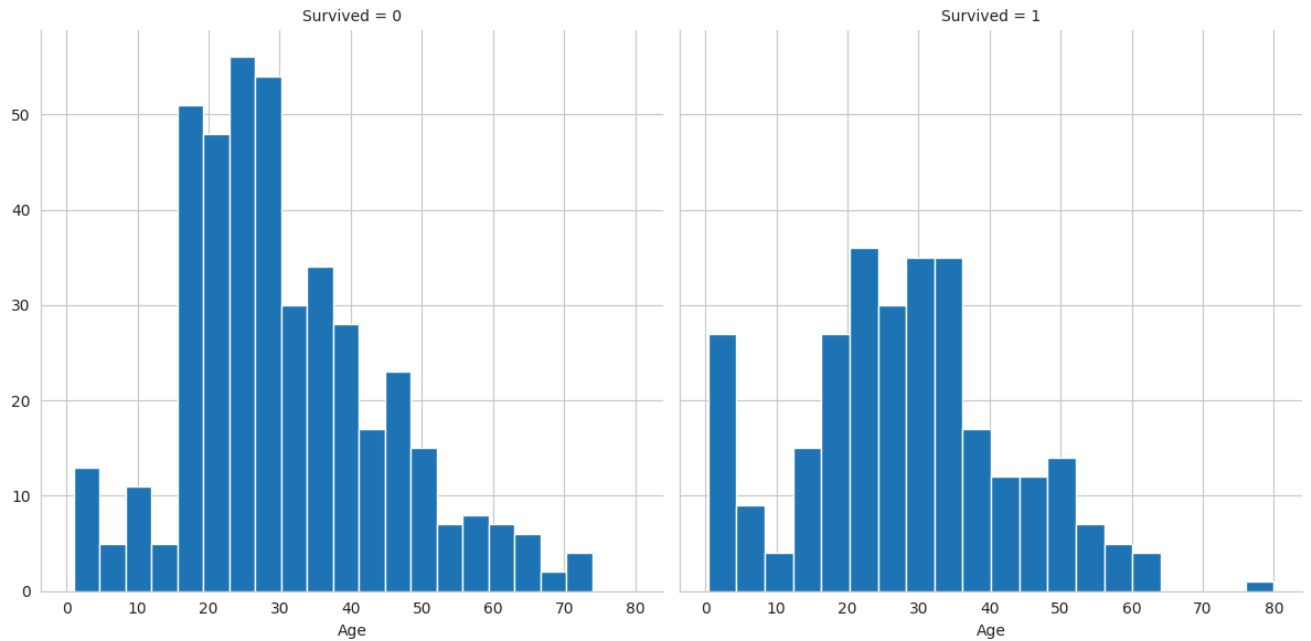


Insight: The pattern is undeniable. A much higher proportion of females survived compared to males. This is another very strong

- Visualization 4: How does age play a role? For a continuous variable like Age, a histogram is a great way to see its distribution.

```
# A FacetGrid allows us to create multiple plots side-by-side to compare distributions.
# Here, we create one histogram for passengers who died (col='Survived'=0) and one for those who survived (col='Survived'=1).
g = sns.FacetGrid(df, col='Survived', height=6)
g.map(plt.hist, 'Age', bins=20)
plt.show()
```

print("Insight: The age distribution for those who did not survive is centered around the 20-40 age range. For those who survived



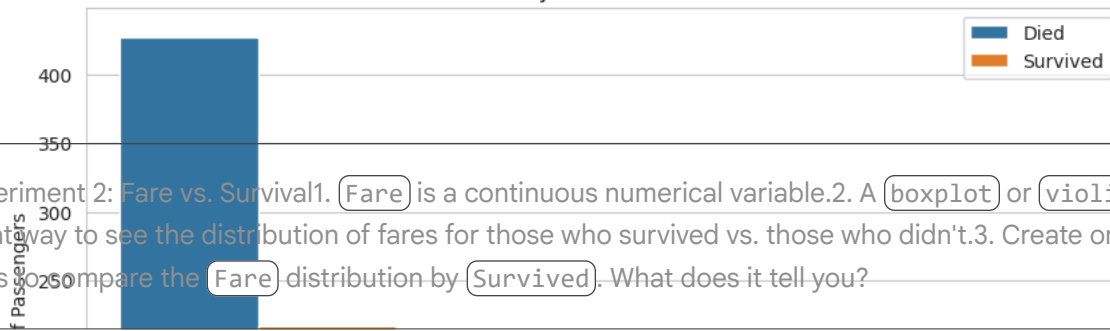
Insight: The age distribution for those who did not survive is centered around the 20-40 age range. For those who survived, the

Part 4: Student Experimentation **Instructions:** Create your own visualizations to explore other relationships in the data.

Experiment 1: Port of Embarkation1. The `Embarked` column tells you where a passenger boarded the ship (C = Cherbourg, Q = Queenstown, S = Southampton).2. Create a `countplot` to see how survival rates differed by the port of embarkation. Does where they boarded seem to be related to their survival?

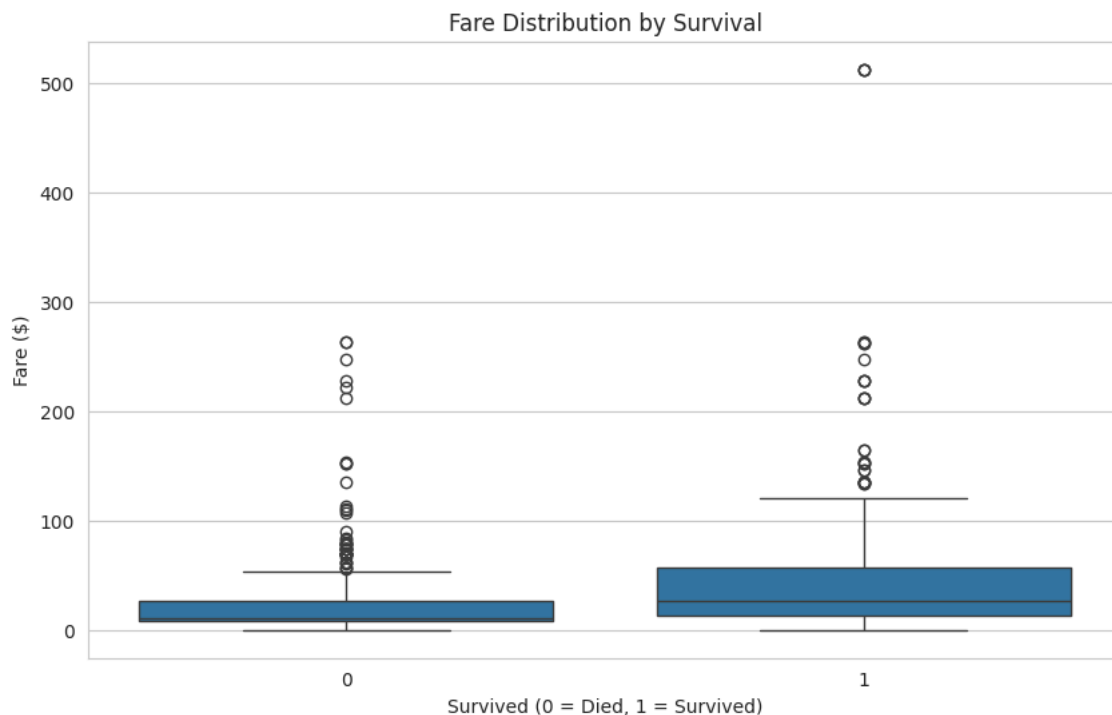
```
plt.figure(figsize=(10, 6))
sns.countplot(x='Embarked', hue='Survived', data=df)
plt.title('Survival by Port of Embarkation')
plt.xlabel('Port of Embarkation')
plt.ylabel('Number of Passengers')
plt.legend(['Died', 'Survived'])
plt.show()
```

Survival by Port of Embarkation



Experiment 2: Fare vs. Survival1. (Fare) is a continuous numerical variable.2. A (boxplot) or (violinplot) is a great way to see the distribution of fares for those who survived vs. those who didn't.3. Create one of these plots to compare the (Fare) distribution by (Survived). What does it tell you?

```
plt.figure(figsize=(10, 6))
sns.boxplot(x='Survived', y='Fare', data=df)
plt.title('Fare Distribution by Survival')
plt.xlabel('Survived (0 = Died, 1 = Survived)')
plt.ylabel('Fare ($)')
plt.show()
```



Knowledge CheckInstructions: Answer the following questions in this markdown cell.

1. What is the primary goal of Exploratory Data Analysis (EDA)?
2. Based on the plots in this lab, what kind of person had the best chance of surviving the Titanic? (Describe them in terms of class, gender, and age).
3. Why is it important to visualize data instead of just looking at summary statistics? What can a plot show you that a number like 'mean' or 'count' can't?

L04 REFLECTIVE JOURNAL

L04_Shareefah_Adeola_ITAI1371.pdf

ITAI 1371 — Module 04: Exploratory Data Analysis

Module 04 Lab – Exploratory Data Analysis

Completed Notebook PDF

Dataset: Titanic | Course Lab

Objective: Explore a dataset to find patterns, anomalies, and insights before modeling using summary statistics and visualizations.

Part 1: Setup and Data Loading

EDA uses summary statistics and visualizations to understand a dataset's main characteristics, including patterns, missing values, errors, and relationships between variables.

Executed setup code

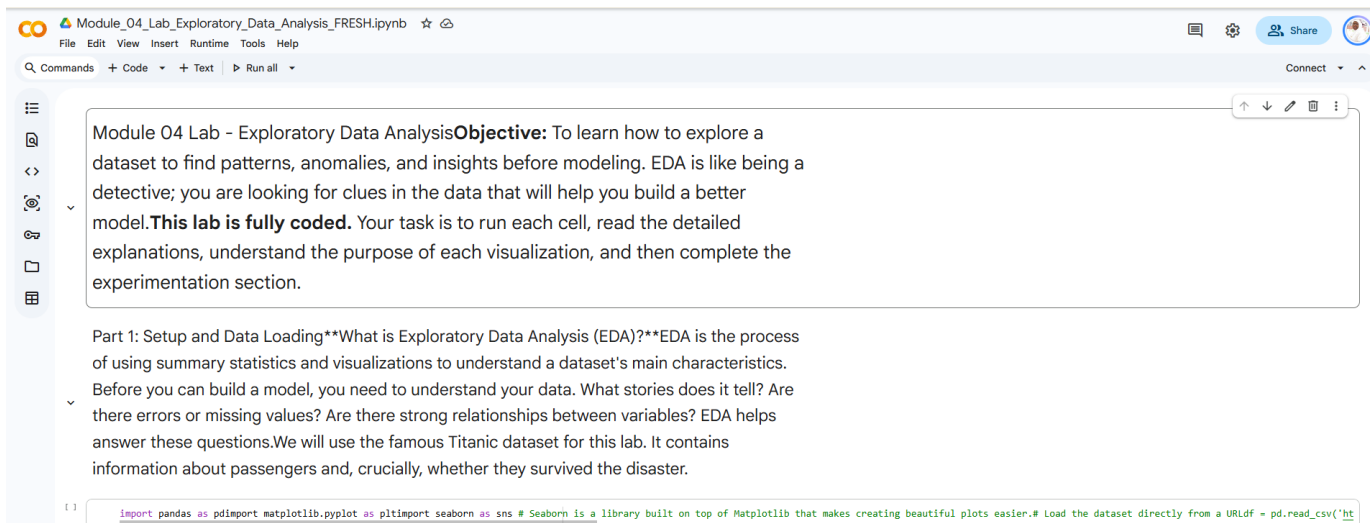
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns # Seaborn is a library built on top of Matplotlib that makes creating beautiful plots easier

# Load the dataset directly from a URL
df = pd.read_csv('https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv')

print("--- First 5 Rows ---")
print(df.head())

print("--- Basic Info ---")
# .info() is a great first command. It tells us the column names, how many non-null values are in each column,
# Notice that 'Age' and 'Cabin' have missing values!
df.info()
```

Colab execution/output evidence



The screenshot shows a Google Colab notebook titled "Module_04_Lab_Exploratory_Data_Analysis_FRESH.ipynb". The notebook interface includes a menu bar (File, Edit, View, Insert, Runtime, Tools, Help) and a toolbar with icons for commands, code, text, and running. The left sidebar shows a file explorer with a folder icon. The main area displays the notebook content, which includes a text block and a code cell.

Module 04 Lab - Exploratory Data Analysis
Objective: To learn how to explore a dataset to find patterns, anomalies, and insights before modeling. EDA is like being a detective; you are looking for clues in the data that will help you build a better model. **This lab is fully coded.** Your task is to run each cell, read the detailed explanations, understand the purpose of each visualization, and then complete the experimentation section.

Part 1: Setup and Data Loading**What is Exploratory Data Analysis (EDA)?**EDA is the process of using summary statistics and visualizations to understand a dataset's main characteristics. Before you can build a model, you need to understand your data. What stories does it tell? Are there errors or missing values? Are there strong relationships between variables? EDA helps answer these questions. We will use the famous Titanic dataset for this lab. It contains information about passengers and, crucially, whether they survived the disaster.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns # Seaborn is a library built on top of Matplotlib that makes creating beautiful plots easier.
# Load the dataset directly from a URL
df = pd.read_csv('ht
```

Part 2: Descriptive Statistics

The `.describe()` method provides a high-level numerical summary, including count, mean, standard deviation, minimum, quartiles, and maximum values for numerical columns.

```
# Get summary statistics for numerical columns
print("--- Descriptive Statistics ---")
print(df.describe())

print("--- Key Insights from Statistics ---")
print(f"The average age of a passenger was {df['Age'].mean():.1f} years.")
print(f"The overall survival rate was {df['Survived'].mean():.1%}.")
print(f"Fares ranged from ${df['Fare'].min()} to a whopping ${df['Fare'].max()}."
```

Execution/output evidence

[2]
✓ 0s

```
# Get summary statistics for numerical columns
print("--- Descriptive Statistics ---")
print(df.describe())

print("--- Key Insights from Statistics ---")
print(f"The average age of a passenger was {df['Age'].mean():.1f} years.")
print(f"The overall survival rate was {df['Survived'].mean():.1%}.")
print(f"Fares ranged from ${df['Fare'].min()} to a whopping ${df['Fare'].max()}."
```

✓

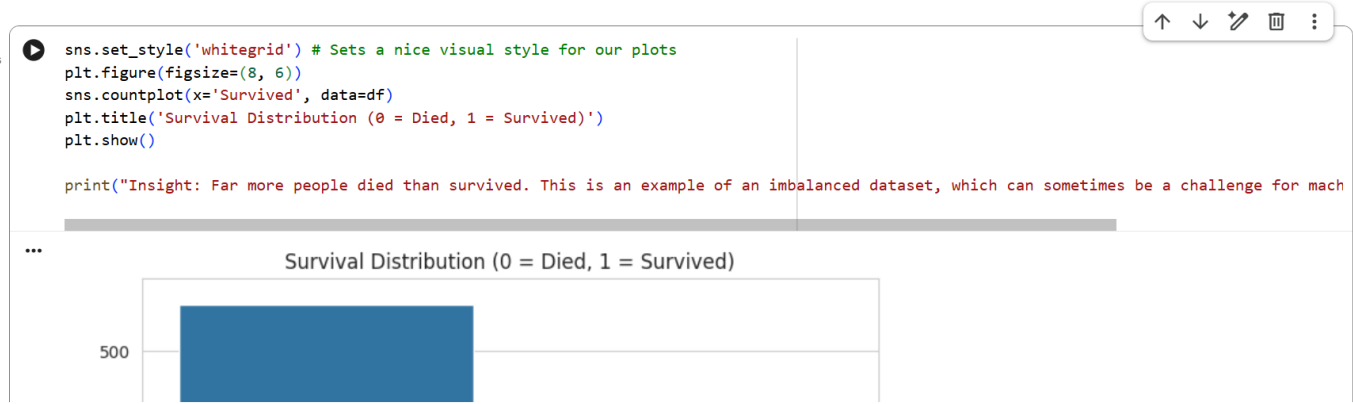
```
... --- Descriptive Statistics ---
      PassengerId  Survived  Pclass     Age  SibSp  \
count    891.000000    891.000000    891.000000  714.000000  891.000000
mean       446.000000     0.383838     2.308642   29.699118     0.523008
std       257.353842     0.486592     0.836071   14.526497     1.102743
min         1.000000     0.000000     1.000000     0.420000     0.000000
25%       223.500000     0.000000     2.000000   20.125000     0.000000
50%       446.000000     0.000000     3.000000   28.000000     0.000000
---
```

Part 3: Visual EDA – Telling Stories with Plots

Visualization 1: Survival Distribution

The countplot shows the distribution of the Survived variable. The lab insight notes that more passengers died than survived.

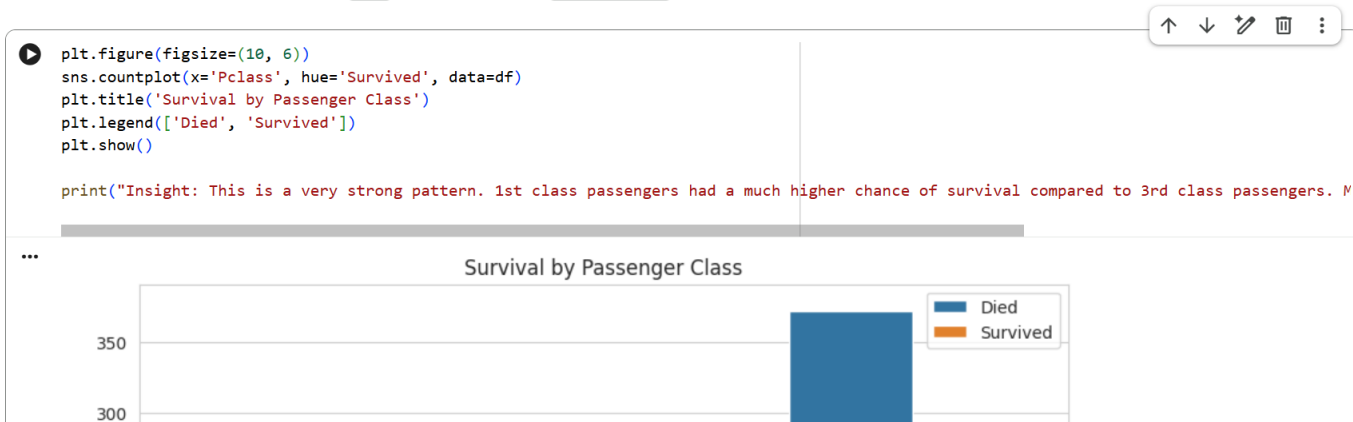
Visualization 1: How many survived? A simple `countplot` is the best way to see the distribution of a categorical variable, like our target `Survived`.



Visualization 2: Survival by Passenger Class

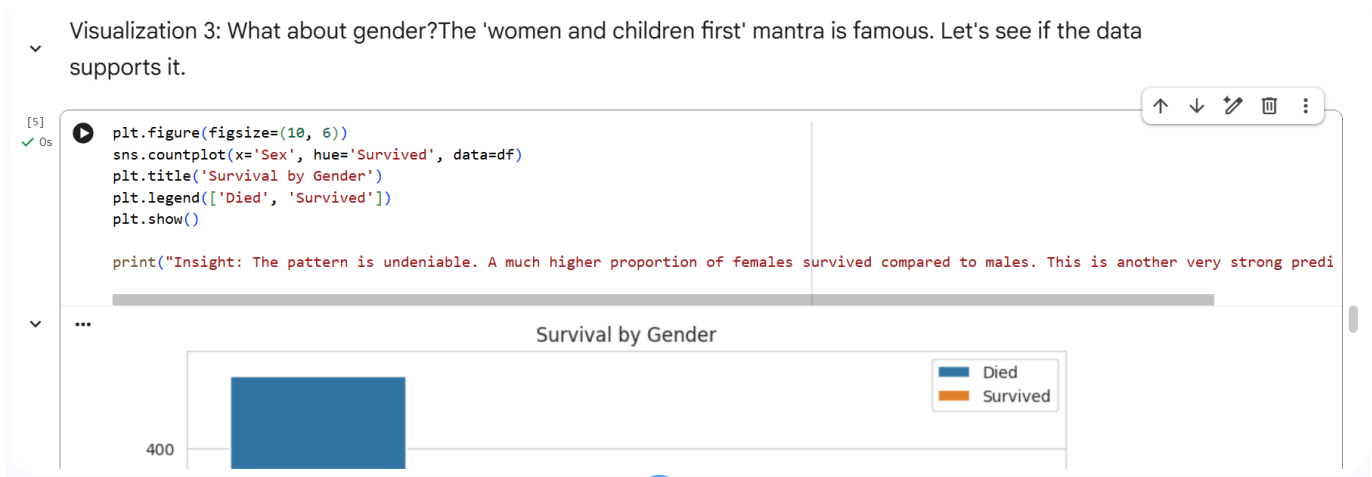
The countplot compares outcomes across passenger classes and shows a strong difference in survival outcomes by class.

Visualization 2: Does passenger class matter for survival? Now we want to see if there's a relationship between two variables. We can use the `hue` parameter in `countplot` to split the bars by another category.



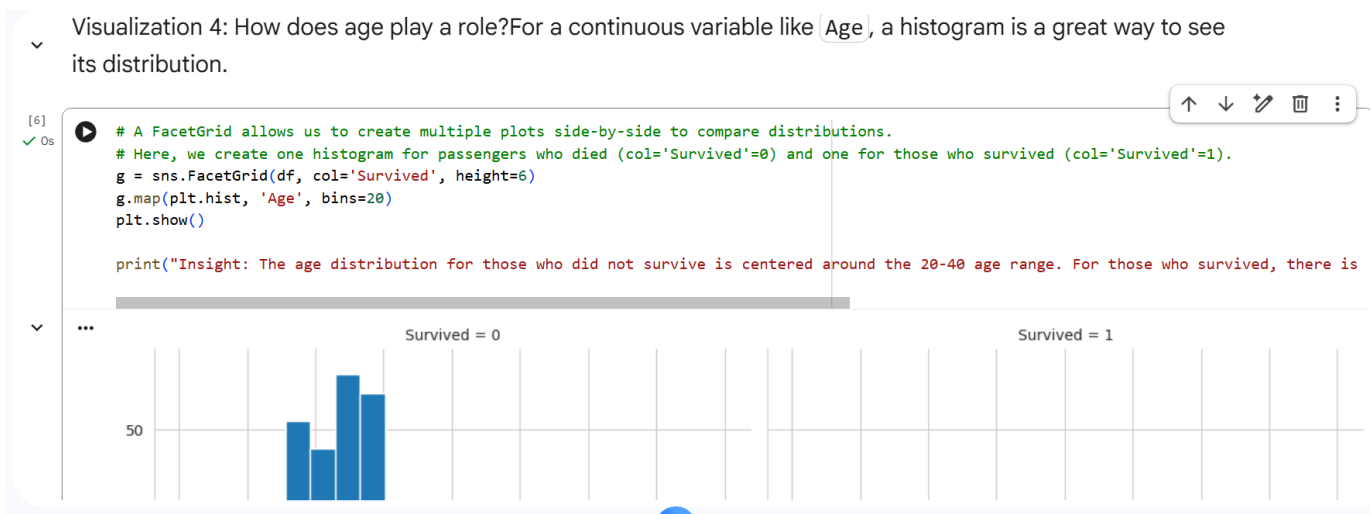
Visualization 3: Survival by Gender

The visualization compares survival outcomes for females and males. The lab insight notes a much higher proportion of females survived compared with males.



Visualization 4: Age Distribution by Survival

The FacetGrid compares age distributions for passengers who died and those who survived. The lab notes a noticeable pattern among younger children who survived.



Part 4: Student Experimentation

Experiment 1: Port of Embarkation

A countplot was created to compare survival outcomes by embarkation port: C = Cherbourg, Q = Queenstown, and S = Southampton.

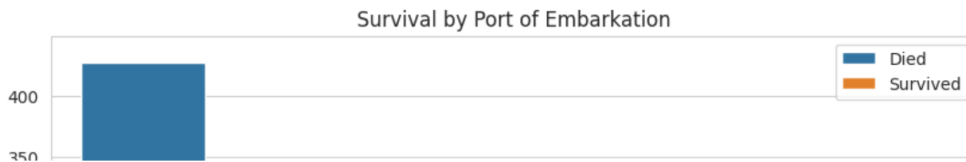
```
plt.figure(figsize=(10, 6))
sns.countplot(x='Embarked', hue='Survived', data=df)
plt.title('Survival by Port of Embarkation')
plt.xlabel('Port of Embarkation')
plt.ylabel('Number of Passengers')
plt.legend(['Died', 'Survived'])
plt.show()
```

Interpretation: The chart shows differences in survival outcomes across the three ports of embarkation. This indicates an association in this dataset, but the plot alone does not establish that the port caused the difference.

Experiment 1: Port of Embarkation1. The `Embarked` column tells you where a passenger boarded the ship (C =

- Cherbourg, Q = Queenstown, S = Southampton).
- 2. Create a `countplot` to see how survival rates differed by the port of embarkation. Does where they boarded seem to be related to their survival?

```
[9] ✓ 0s
plt.figure(figsize=(10, 6))
sns.countplot(x='Embarked', hue='Survived', data=df)
plt.title('Survival by Port of Embarkation')
plt.xlabel('Port of Embarkation')
plt.ylabel('Number of Passengers')
plt.legend(['Died', 'Survived'])
plt.show()
```



Experiment 2: Fare vs. Survival

A boxplot was used to compare fare distributions for passengers who died versus those who survived.

```
plt.figure(figsize=(10, 6))
sns.boxplot(x='Survived', y='Fare', data=df)
plt.title('Fare Distribution by Survival')
plt.xlabel('Survived (0 = Died, 1 = Survived)')
plt.ylabel('Fare')
plt.show()
```

Interpretation: The boxplot suggests that passengers who paid higher fares generally had higher survival outcomes than passengers who paid lower fares. The plot shows an association and does not establish causation.

Experiment 2: Fare vs. Survival1. `Fare` is a continuous numerical variable.2. A `boxplot` or `violinplot` is a

- great way to see the distribution of fares for those who survived vs. those who didn't.3. Create one of these plots to compare the `Fare` distribution by `Survived`. What does it tell you?

```
[10]
✓ 0s
plt.figure(figsize=(10, 6))
sns.boxplot(x='Survived', y='Fare', data=df)
plt.title('Fare Distribution by Survival')
plt.xlabel('Survived (0 = Died, 1 = Survived)')
plt.ylabel('Fare')
plt.show()
```



Knowledge Check

1. What is the primary goal of Exploratory Data Analysis (EDA)?

The primary goal of Exploratory Data Analysis (EDA) is to understand the main characteristics of a dataset by using summary statistics and visualizations. It helps identify patterns, relationships, errors, and missing values before building a model.

2. Based on the plots, what kind of person had the best chance of surviving the Titanic?

Based on the plots, a passenger in 1st class, female, and younger in age, particularly a child, had a better chance of surviving the Titanic. The plots showed that passenger class and gender were strongly related to survival, and the age distribution showed a noticeable pattern among children who survived.

3. Why is it important to visualize data instead of just looking at summary statistics?

Visualizing data is important because a plot can show patterns, relationships, differences, distributions, and outliers that may not be obvious from summary statistics alone. A mean or count gives us a single number, while a plot allows us to see how the data is distributed and how variables relate to each other.

Notebook completion evidence



Knowledge CheckInstructions: Answer the following questions in this markdown cell.1.

What is the primary goal of Exploratory Data Analysis (EDA)?2. Based on the plots in this lab, what kind of person had the best chance of surviving the Titanic? (Describe them in terms of class, gender, and age).3. Why is it important to visualize data instead of just looking at summary statistics? What can a plot show you that a number like 'mean' or 'count' can't?[ENTER YOUR ANSWERS HERE]

L04 REFLECTIVE JOURNAL

L04_YilinLeng_ITAI1371.pdf

ITAI 1371 — Module 04: Exploratory Data Analysis

Module 04 Lab - Exploratory Data Analysis **Objective:**

- To learn how to explore a dataset to find patterns, anomalies, and insights before modeling. EDA is like being a detective; you are looking for clues in the data that will help you build a better model. **This lab is fully coded.** Your task is to run each cell, read the detailed explanations, understand the purpose of each visualization, and then complete the experimentation section.

Part 1: Setup and Data Loading**What is Exploratory Data

Analysis (EDA)?**EDA is the process of using summary

statistics and visualizations to understand a dataset's main

characteristics. Before you can build a model, you need to

understand your data. What stories does it tell? Are there

errors or missing values? Are there strong relationships

between variables? EDA helps answer these questions. We will

use the famous Titanic dataset for this lab. It contains

information about passengers and, crucially, whether they

survived the disaster.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns # Seaborn is a library built on top of Matplotlib that n

# Load the dataset directly from a URL
df = pd.read_csv('https://raw.githubusercontent.com/datasciencedojo/datasets/m
```

```
print("--- First 5 Rows ---")
print(df.head())
print("--- Basic Info ---")
df.info()
```

```
--- First 5 Rows ---
```

	PassengerId	Survived	Pclass	\
0	1	0	3	
1	2	1	1	
2	3	1	3	
3	4	1	1	
4	5	0	3	

	Name	Sex	Age	SibSp	\
0	Braund, Mr. Owen Harris	male	22.0	1	
1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	
2	Heikkinen, Miss. Laina	female	26.0	0	
3	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	
4	Allen, Mr. William Henry	male	35.0	0	

	Parch	Ticket	Fare	Cabin	Embarked
0	0	A/5 21171	7.2500	NaN	S
1	0	PC 17599	71.2833	C85	C
2	0	STON/O2. 3101282	7.9250	NaN	S
3	0	113803	53.1000	C123	S
4	0	373450	8.0500	NaN	S

```
--- Basic Info ---
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 891 entries, 0 to 890
```

```
Data columns (total 12 columns):
```

#	Column	Non-Null Count	Dtype
0	PassengerId	891 non-null	int64
1	Survived	891 non-null	int64
2	Pclass	891 non-null	int64
3	Name	891 non-null	object
4	Sex	891 non-null	object
5	Age	714 non-null	float64
6	SibSp	891 non-null	int64
7	Parch	891 non-null	int64
8	Ticket	891 non-null	object
9	Fare	891 non-null	float64
10	Cabin	204 non-null	object
11	Embarked	889 non-null	object

```
dtypes: float64(2), int64(5), object(5)
```

```
memory usage: 83.7+ KB
```

Part 2: Descriptive Statistics Let's start by getting a high-level numerical summary of the data. The `.describe()` method is

perfect for this. It calculates statistics like mean, standard deviation, min, and max for the numerical columns.

```
# Get summary statistics for numerical columnsprint("--- Descriptive Statistics
```

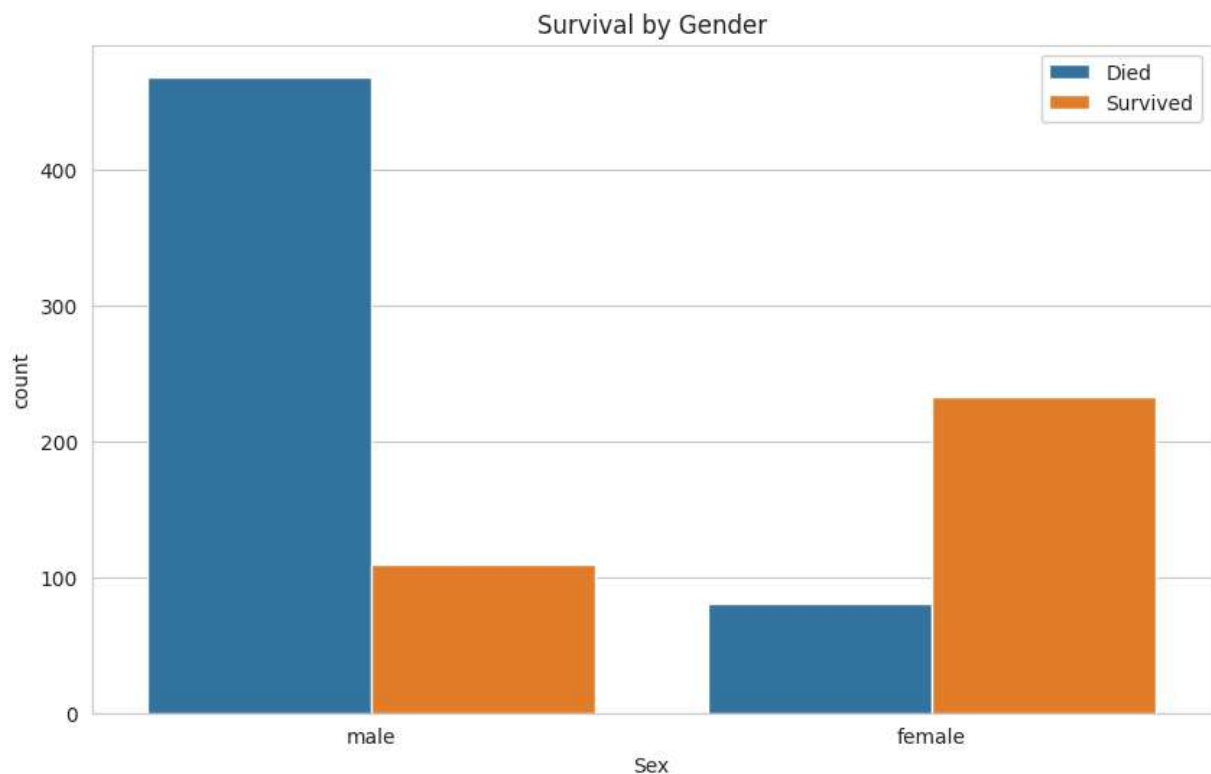
Part 3: Visual EDA - Telling Stories with PlotsNumbers are great, but plots make patterns and relationships immediately obvious. The goal of visual EDA is to turn data into insights.**A Note on Plotting Libraries:*** **Matplotlib:** The foundational library, gives you full control over everything.* **Seaborn:** Built on Matplotlib, it provides a simpler, high-level interface for creating common statistical plots. We will use Seaborn for its ease of use and attractive defaults.

Visualization 1: How many survived?A simple `countplot` is the best way to see the distribution of a categorical variable, like our target `Survived`.

```
sns.set_style('whitegrid') # Sets a nice visual style for our plotsplt.figure(f
```

Visualization 2: Does passenger class matter for survival?Now we want to see if there's a relationship between two variables. We can use the `hue` parameter in `countplot` to split the bars by another category.

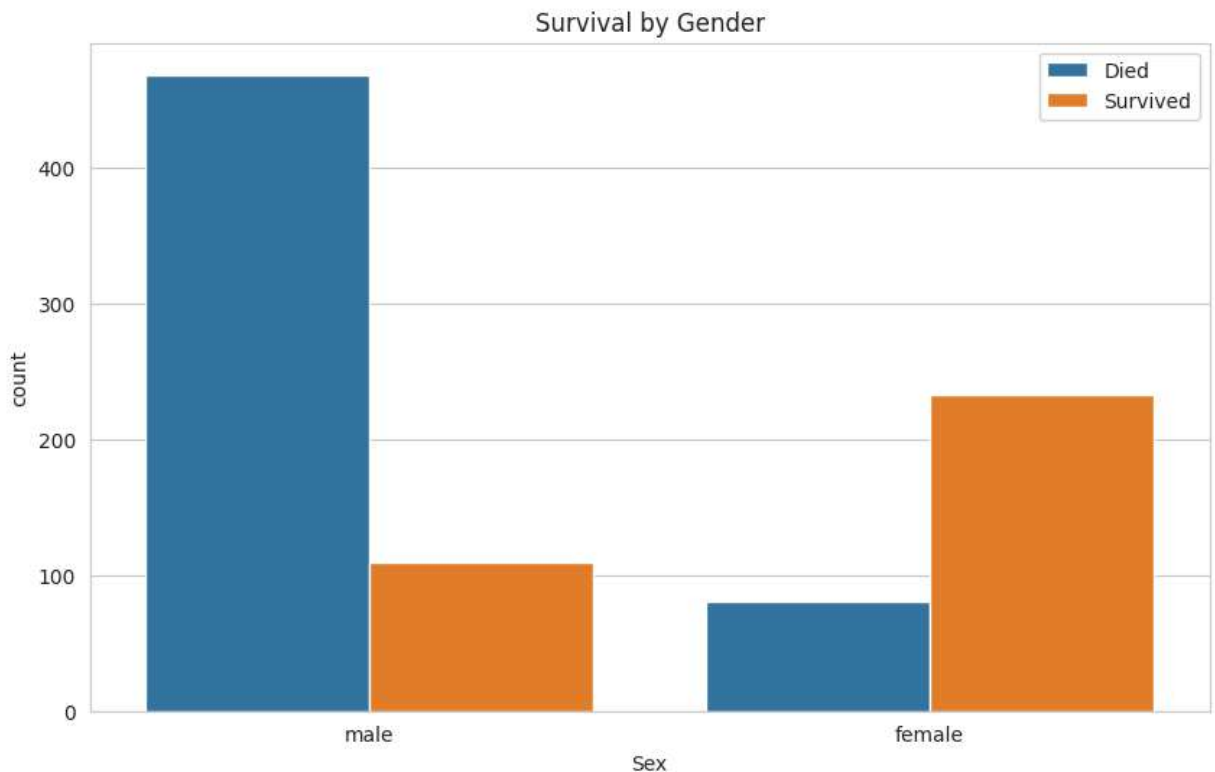
```
plt.figure(figsize=(10, 6))
sns.countplot(x='Sex', hue='Survived', data=df)
plt.title('Survival by Gender')
plt.legend(['Died', 'Survived'])
plt.show()
print("Insight: The pattern is undeniable. A much higher proportion of females
```



Insight: The pattern is undeniable. A much higher proportion of females survived

✓ Visualization 3: What about gender?The 'women and children first' mantra is famous. Let's see if the data supports it.

```
plt.figure(figsize=(10, 6))
sns.countplot(x='Sex', hue='Survived', data=df)
plt.title('Survival by Gender')
plt.legend(['Died', 'Survived'])
plt.show()
print("Insight: The pattern is undeniable. A much higher proportion of females
```



Insight: The pattern is undeniable. A much higher proportion of females survived

- Visualization 4: How does age play a role? For a continuous variable like **Age**, a histogram is a great way to see its distribution.

```
# A FacetGrid allows us to create multiple plots side-by-side to compare distri
```

- Part 4: Student Experimentation **Instructions:** Create your own visualizations to explore other relationships in the data.

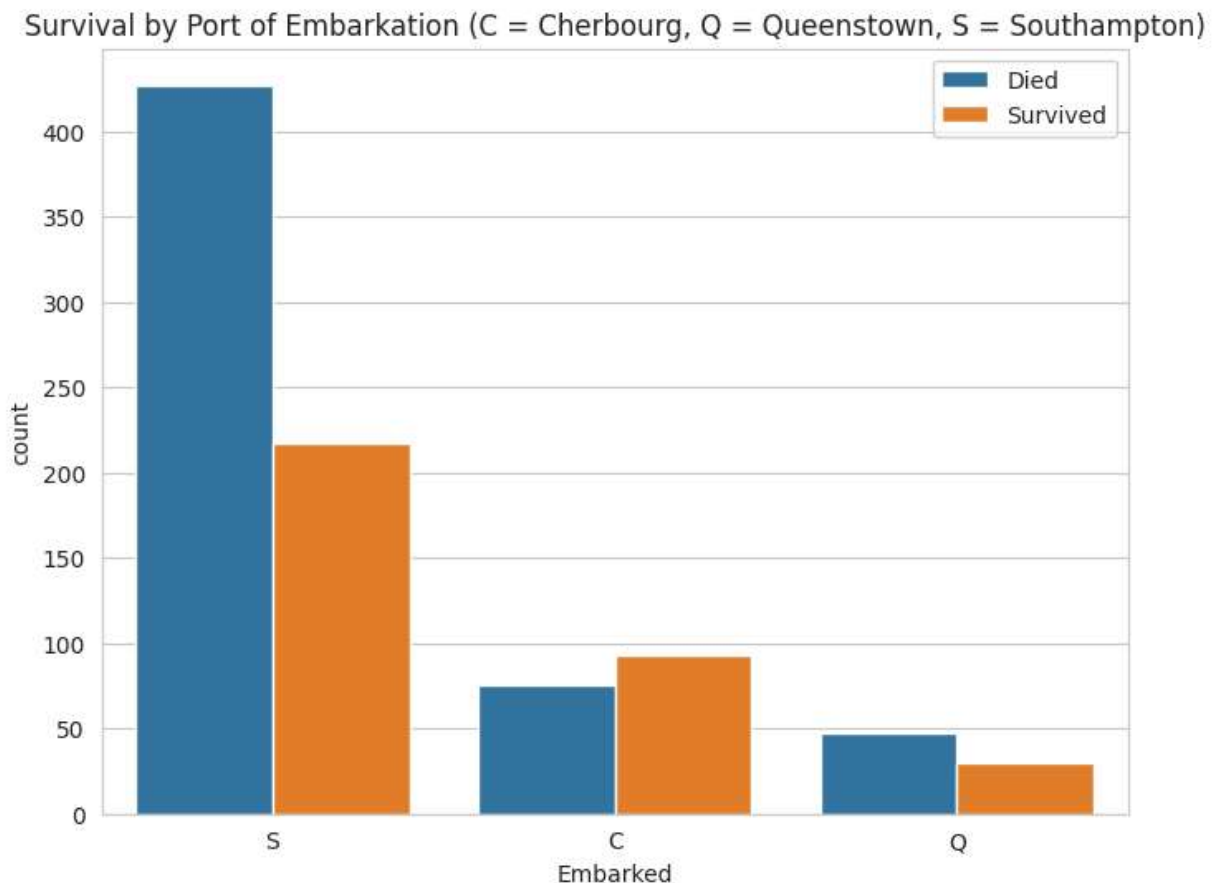
Experiment 1: Port of Embarkation. The **Embarked** column tells you where a passenger boarded the ship (C = Cherbourg, Q =

- Queenstown, S = Southampton). 2. Create a **countplot** to see how

survival rates differed by the port of embarkation. Does where they boarded seem to be related to their survival?

```
# --- ENTER YOUR CODE HERE ---
plt.figure(figsize=(8, 6))
sns.countplot(x='Embarked', hue='Survived', data=df)
plt.title('Survival by Port of Embarkation (C = Cherbourg, Q = Queenstown, S = Southampton)')
plt.legend(['Died', 'Survived'])
plt.show()

print("Insight: Passengers who boarded at Cherbourg (C) had a higher proportion of survival")
```



Insight: Passengers who boarded at Cherbourg (C) had a higher proportion of survival

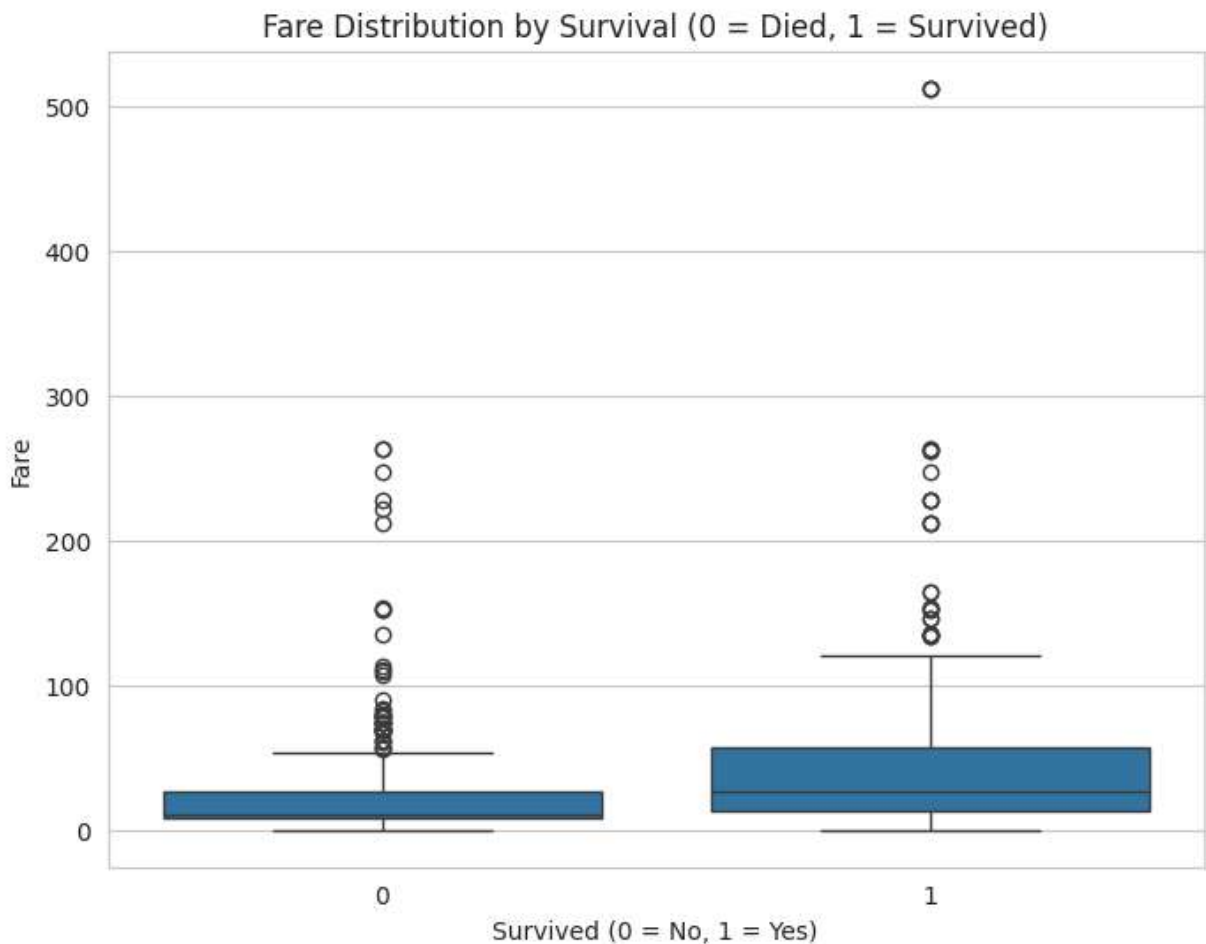
Experiment 2: Fare vs. Survival1. Fare is a continuous numerical variable.2. A boxplot or violinplot is a great way to see the

✓ distribution of fares for those who survived vs. those who didn't.3.

Create one of these plots to compare the **Fare** distribution by **Survived**. What does it tell you?

```
# --- ENTER YOUR CODE HERE ---
plt.figure(figsize=(8, 6))
sns.boxplot(x='Survived', y='Fare', data=df)
plt.title('Fare Distribution by Survival (0 = Died, 1 = Survived)')
plt.xlabel('Survived (0 = No, 1 = Yes)')
plt.ylabel('Fare')
plt.show()

print("Insight: Survivors generally paid a higher ticket fare compared to those
```



Insight: Survivors generally paid a higher ticket fare compared to those who did not survive.



Knowledge Check Instructions: Answer the following

questions in this markdown cell.1. ****What is the primary goal**

of Exploratory Data Analysis (EDA)?**2. Based on the plots in this lab, what kind of person had the best chance of surviving the Titanic? (Describe them in terms of class, gender, and age).****3. Why is it important to visualize data instead of just looking at summary statistics? What can a plot show you that a number like 'mean' or 'count' can't?]**

1.The main goal of Exploratory Data Analysis (EDA) is to understand the dataset before building a Machine Learning model. It helps us find patterns, missing values, and possible relationships between variables.

2.Based on the charts, female passengers in first class, especially children and younger passengers, had a higher chance of surviving the Titanic disaster. The charts show clear differences in survival between genders and age groups. They also show that first class passengers had higher survival rates.

3.Visualization is important because it helps us see patterns that numbers alone may not show clearly. For example, the `describe()` function gives information like the mean and count, but charts can show differences between groups and how the data is distributed. This makes the data easier to understand.]

L04 REFLECTIVE JOURNAL

LO4_KeithJones_ITAI1371.pdf

ITAI 1371 — Module 04: Exploratory Data Analysis

Module 04 Lab - Exploratory Data Analysis **Objective:** To learn how to explore a dataset to find patterns, anomalies, and insights before modeling. EDA is like being a detective; you are looking for clues in the data that will help you build a better model. **This lab is fully coded.** Your task is to run each cell, read the detailed explanations, understand the purpose of each visualization, and then complete the experimentation section.

Part 1: Setup and Data Loading ****What is Exploratory Data Analysis (EDA)?**** EDA is the process of using summary statistics and visualizations to understand a dataset's main characteristics. Before you can build a model, you need to understand your data. What stories does it tell? Are there errors or missing values? Are there strong relationships between variables? EDA helps answer these questions. We will use the famous Titanic dataset for this lab. It contains information about passengers and, crucially, whether they survived the disaster.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns # Seaborn is a library built on top of Matplotlib that makes creating beautiful plots easier.
# Load the dataset directly from a URL
df = pd.read_csv('https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv')
print("---- First 5 Rows ----")
print(df.head())
print("---- Basic Info ----")
# .info() is a great first command. It tells us the column names, how many non-null values are in each column, and t
# Notice that 'Age' and 'Cabin' have missing values!
df.info()
```

```
---- First 5 Rows ----
   PassengerId  Survived  Pclass  \
0             1         0       3
1             2         1       1
2             3         1       3
3             4         1       1
4             5         0       3

   Name                               Sex  Age  SibSp  \
0   Braund, Mr. Owen Harris             male  22.0      1
1  Cumings, Mrs. John Bradley (Florence Briggs Th... female  38.0      1
2   Heikkinen, Miss. Laina              female  26.0      0
3  Futrelle, Mrs. Jacques Heath (Lily May Peel)    female  35.0      1
4    Allen, Mr. William Henry             male  35.0      0

   Parch  Ticket   Fare Cabin Embarked
0      0   A/5 21171   7.2500   NaN      S
1      0    PC 17599  71.2833   C85      C
2      0  STON/O2. 3101282   7.9250   NaN      S
3      0   113803   53.1000  C123      S
4      0   373450   8.0500   NaN      S

---- Basic Info ----
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
#   Column      Non-Null Count  Dtype
---  -
0   PassengerId  891 non-null    int64
1   Survived     891 non-null    int64
2   Pclass       891 non-null    int64
3   Name         891 non-null    object
4   Sex          891 non-null    object
5   Age         714 non-null    float64
6   SibSp        891 non-null    int64
7   Parch        891 non-null    int64
8   Ticket       891 non-null    object
9   Fare         891 non-null    float64
10  Cabin        204 non-null    object
```

```
11 Embarked      889 non-null object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
```

Part 2: Descriptive Statistics Let's start by getting a high-level numerical summary of the data.

- ✓ The `.describe()` method is perfect for this. It calculates statistics like mean, standard deviation, min, and max for the numerical columns.

```
# Get summary statistics for numerical columns
print("---- Descriptive Statistics ----")
print(df.describe())
print("---- Key Insights from Statistics ----")
print(f"The average age of a passenger was {df['Age'].mean():.1f} years.")
print(f"The overall survival rate was {df['Survived'].mean():.1%}.")
print(f"Fares ranged from ${df['Fare'].min()} to a whopping ${df['Fare'].max()}.")
```

```
---- Descriptive Statistics ----
count    PassengerId   Survived  Pclass     Age    SibSp  \
mean     446.000000    0.383838    2.308642  29.699118  0.523008
std       257.353842    0.486592    0.836071  14.526497  1.102743
min        1.000000    0.000000    1.000000   0.420000  0.000000
25%       223.500000    0.000000    2.000000  20.125000  0.000000
50%       446.000000    0.000000    3.000000  28.000000  0.000000
75%       668.500000    1.000000    3.000000  38.000000  1.000000
max       891.000000    1.000000    3.000000  80.000000  8.000000

count    Parch    Fare
mean     0.381594  32.204208
std       0.806057  49.693429
min       0.000000   0.000000
25%       0.000000   7.910400
50%       0.000000  14.454200
75%       0.000000  31.000000
max       6.000000  512.329200

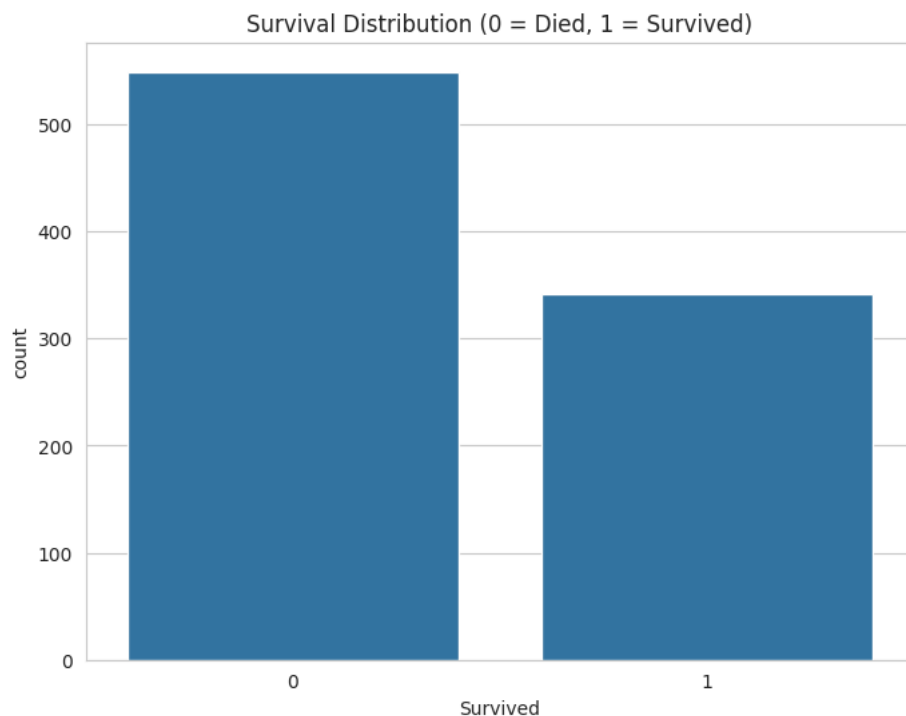
---- Key Insights from Statistics ----
The average age of a passenger was 29.7 years.
The overall survival rate was 38.4%.
Fares ranged from $0.0 to a whopping $512.3292.
```

Part 3: Visual EDA - Telling Stories with Plots Numbers are great, but plots make patterns and relationships immediately obvious. The goal of visual EDA is to turn data into insights. **A Note on**

- ✓ **Plotting Libraries:**
 - * **Matplotlib:** The foundational library, gives you full control over everything.
 - * **Seaborn:** Built on Matplotlib, it provides a simpler, high-level interface for creating common statistical plots. We will use Seaborn for its ease of use and attractive defaults.

Visualization 1: How many survived? A simple `countplot` is the best way to see the distribution of a categorical variable, like our target `Survived`.

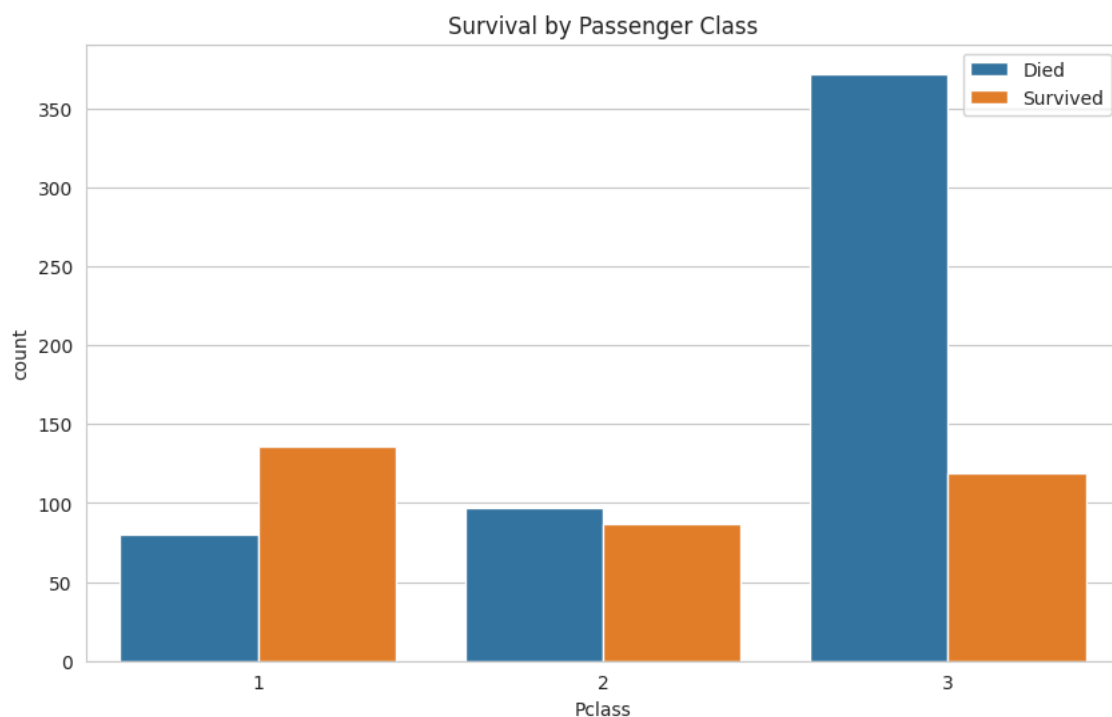
```
sns.set_style('whitegrid') # Sets a nice visual style for our plots
plt.figure(figsize=(8, 6))
sns.countplot(x='Survived', data=df)
plt.title('Survival Distribution (0 = Died, 1 = Survived)')
plt.show()
print("Insight: Far more people died than survived. This is an example of an imbalanced dataset, which can sometimes")
```



Insight: Far more people died than survived. This is an example of an imbalanced dataset, which can sometimes be a ch

Visualization 2: Does passenger class matter for survival? Now we want to see if there's a relationship between two variables. We can use the `hue` parameter in `countplot` to split the bars by another category.

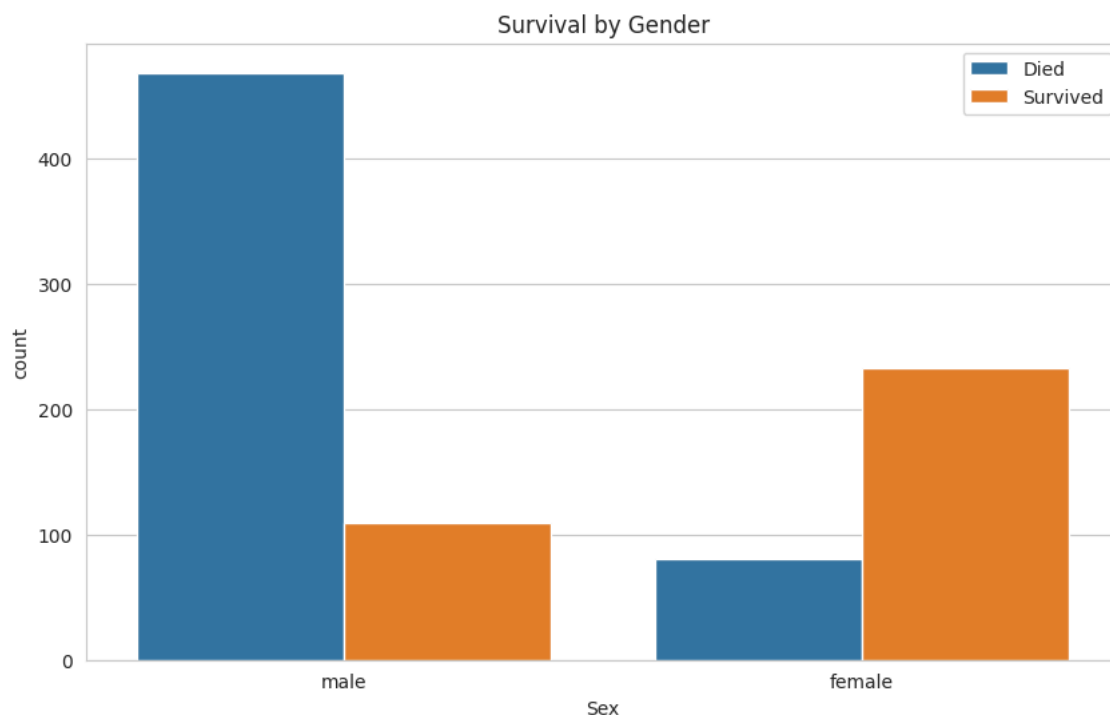
```
plt.figure(figsize=(10, 6))
sns.countplot(x='Pclass', hue='Survived', data=df)
plt.title('Survival by Passenger Class')
plt.legend(['Died', 'Survived'])
plt.show()
print("Insight: This is a very strong pattern. 1st class passengers had a much higher chance of survival compared to :
```



Insight: This is a very strong pattern. 1st class passengers had a much higher chance of survival compared to 3rd cla

- Visualization 3: What about gender?The 'women and children first' mantra is famous. Let's see if the data supports it.

```
plt.figure(figsize=(10, 6))
sns.countplot(x='Sex', hue='Survived', data=df)
plt.title('Survival by Gender')
plt.legend(['Died', 'Survived'])
plt.show()
print("Insight: The pattern is undeniable. A much higher proportion of females survived compared to males. This is a
```

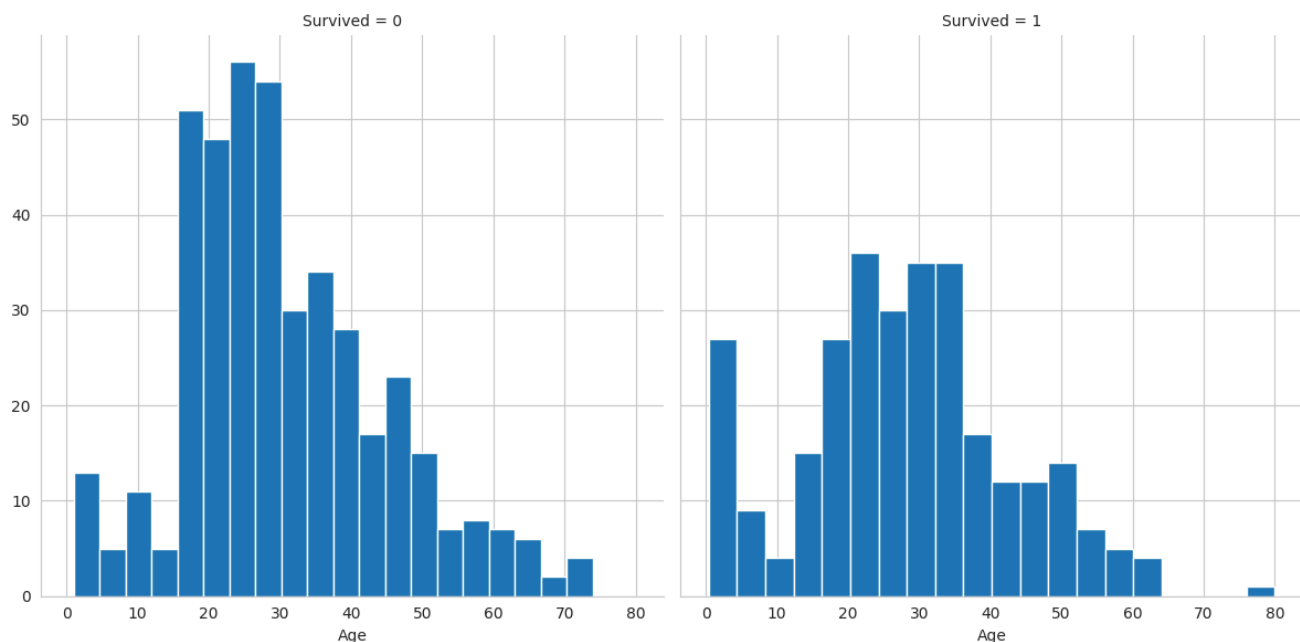


Insight: The pattern is undeniable. A much higher proportion of females survived compared to males. This is another v

- Visualization 4: How does age play a role?For a continuous variable like `Age`, a histogram is a great way to see its distribution.

```
# A FacetGrid allows us to create multiple plots side-by-side to compare distributions.# Here, we create one histogram
# A FacetGrid allows us to create multiple plots side-by-side to compare distributions.
# Here, we create one histogram for passengers who died (col='Survived'=0)
# and one for those who survived (col='Survived'=1).
g = sns.FacetGrid(df, col='Survived', height=6)
g.map(plt.hist, 'Age', bins=20)
plt.show()

print("Insight: The age distribution for those who did not survive is centered around "
      "the 20-40 age range. For those who survived, there is a noticeable spike for "
      "young children. This supports the 'children' part of the mantra.")
```



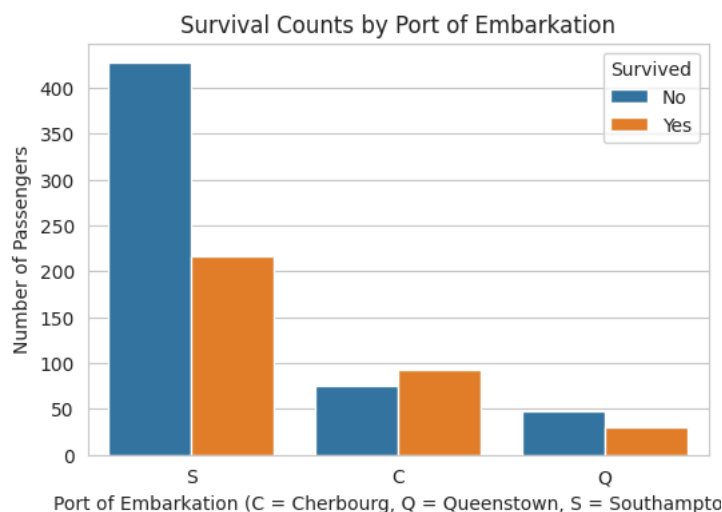
Insight: The age distribution for those who did not survive is centered around the 20–40 age range. For those who sur

Part 4: Student Experimentation **Instructions:** Create your own visualizations to explore other relationships in the data.

Experiment 1: Port of Embarkation1. The `Embarked` column tells you where a passenger boarded the ship (C = Cherbourg, Q = Queenstown, S = Southampton).2. Create a `countplot` to see how survival rates differed by the port of embarkation. Does where they boarded seem to be related to their survival?

```
# import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(6, 4))
sns.countplot(data=df, x='Embarked', hue='Survived')
plt.title('Survival Counts by Port of Embarkation')
plt.xlabel('Port of Embarkation (C = Cherbourg, Q = Queenstown, S = Southampton)')
plt.ylabel('Number of Passengers')
plt.legend(title='Survived', labels=['No', 'Yes'])
plt.show()
```

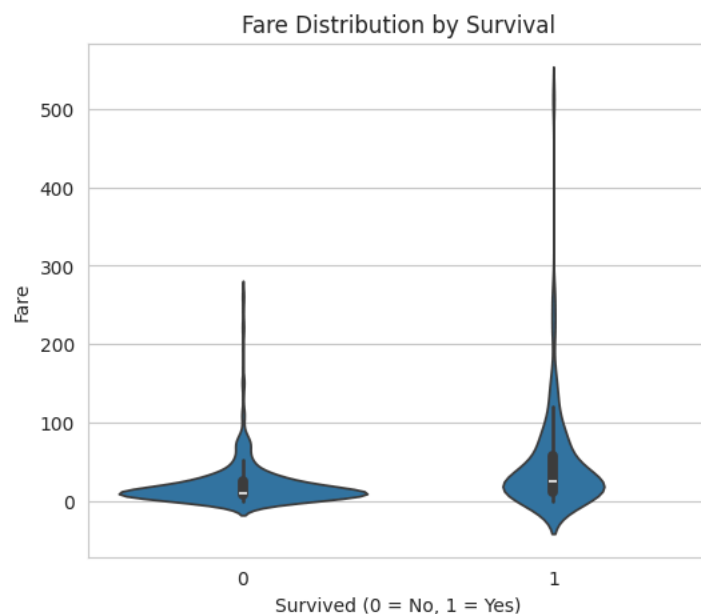
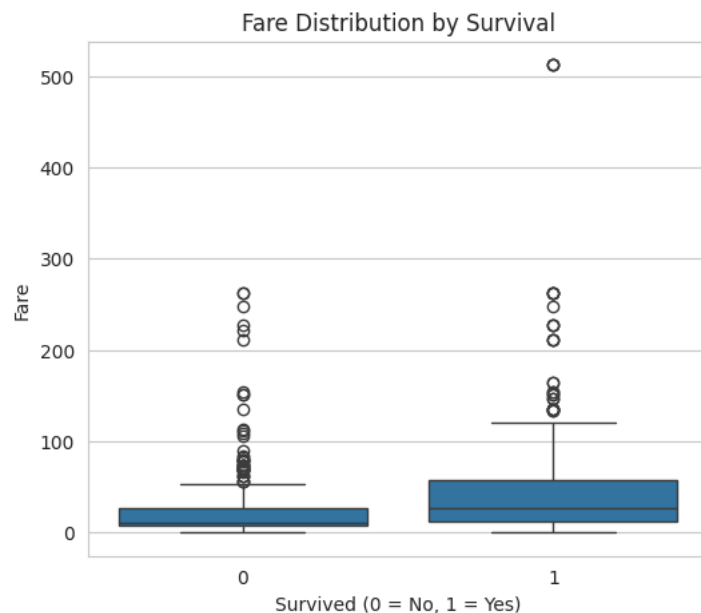


Experiment 2: Fare vs. Survival1. `Fare` is a continuous numerical variable.2. A `boxplot` or `violinplot` is a great way to see the distribution of fares for those who survived vs. those who didn't.3. Create one of these plots to compare the `Fare` distribution by `Survived`. What does it tell you?

```
# import seaborn as sns
import matplotlib.pyplot as plt

# Boxplot
plt.figure(figsize=(6, 5))
sns.boxplot(data=df, x='Survived', y='Fare')
plt.title('Fare Distribution by Survival')
plt.xlabel('Survived (0 = No, 1 = Yes)')
plt.ylabel('Fare')
plt.show()

# Violinplot (shows the full distribution shape, not just quartiles)
plt.figure(figsize=(6, 5))
sns.violinplot(data=df, x='Survived', y='Fare')
plt.title('Fare Distribution by Survival')
plt.xlabel('Survived (0 = No, 1 = Yes)')
plt.ylabel('Fare')
plt.show()
```





Knowledge CheckInstructions: Answer the following questions in this markdown cell.1. **What is the primary goal of Exploratory Data Analysis (EDA)?**2. **Based on the plots in this lab, what kind of person had the best chance of surviving the Titanic? (Describe them in terms of class, gender, and age).**3. **Why is it important to visualize data instead of just looking at summary statistics? What can a plot show you that a number like 'mean' or 'count' can't?**[ENTER YOUR ANSWERS HERE]

Reflection Questions

1. What is the primary goal of Exploratory Data Analysis (EDA)?

The primary goal of EDA is to understand the structure, patterns, and quality of a dataset *before* building any model on it. That means uncovering relationships between variables (like class and survival), spotting anomalies or missing data, checking distributions (like the skew in `Fare`), and forming hypotheses about what might predict the outcome you care about. EDA is exploratory, not confirmatory — it's about generating good questions and catching problems early, not proving a final conclusion.

2. Based on the plots in this lab, what kind of person had the best chance of surviving the Titanic?

The passenger with the best odds of survival was a **female, in 1st class, who was a young child or in early adulthood**. Class mattered enormously — 1st class passengers had much higher survival rates than 3rd class, consistent with the fare distributions we saw (higher fares, tied to class, associated with higher survival). Gender mattered even more starkly, with women surviving at far higher rates than men, reflecting the "women and children first" evacuation norm. Age added a further edge at the youngest end — the FacetGrid histogram showed a distinct spike in survival among young children, while adults aged roughly 20–40 made up the bulk of those who didn't survive.

3. Why is it important to visualize data instead of just looking at summary statistics? What can a plot show you that a number like 'mean' or 'count' can't?

Summary statistics compress a whole distribution into a single number, and that compression can hide exactly the thing you need to see. A mean fare doesn't reveal that the distribution is heavily right-skewed with a long tail of expensive 1st class tickets — a plot shows that